Python vs Java

- Python is more flexible and concise with fewer syntactical rules, while Java is more strict with explicit type declarations and structure.

Feature	Python	Java
Typing	Dynamic	Static
Variable Declaration	No type declaration	Explicit type declaration
Semicolons	Not required	Required
Code Blocks	Indentation	Braces `{}`
Conditionals	No parentheses in `if` conditions	Parentheses required
Loops	`for` and `while`	C-style `for` and `while`
Function Definition	`def` keyword, no return type	Explicit return type
Class Definition	Simpler, no access modifiers	Explicit access modifiers
Inheritance	Parentheses for parent class	`extends` and `implements` keywords
Access Modifiers	No explicit keywords, convention-based	`public`, `private`, `protected`
Arrays/Lists	Dynamic `list`	Fixed-size `Array`, dynamic `ArrayList`
Exception Handling	`try-except`	`try-catch`
Importing	`import`	`import`
Main Method	Not required	Required `main()` method

Typing

	Python	Java
Description	Dynamically typed (no need to declare variable types explicitly)	Statically typed (requires explicit type declaration for variables)

Sample code	`# No type declaration`
	`x = 10`
	`y = "Hello"`
		`// Explicit type declaration`
		`int x = 10;`
		`String y = "Hello";`

Variable Declaration

	Python	Java
Description	Variables are declared by assignment and can change types dynamically	Variables must be declared with a fixed type and cannot change types

Sample code	`# Integer`
	`x = 10`
	`# Now a string`
	`x = "Hello"`
		`// Integer`
		`int x = 10;`
		`//Error, incompatible types`
		`x = "Hello";`

Semicolons

	Python	Java
Description	No semicolons required at the end of statements	Semicolons are required at the end of statements
Sample code	`print("Hello")`	`System.out.println("Hello");`

Indentation and Code Blocks

	Python	Java
Description	Uses indentation (whitespace) to define blocks of code	Uses braces {} to define blocks of code, with indentation as a convention but not enforced

Sample code	`if x > 5:`
	`    print("x is greater than 5")`
		`if (x > 5) {`
		`    System.out.println("x is greater than 5");`
		`}`

Conditionals

	Python	Java
Description	No parentheses required around the condition in if statements	Parentheses are required around the condition in if statements.

Sample code	`if x > 5:`
	`  print("x is greater than 5")`
		`if (x > 5) {`
		`  System.out.println("x is greater than 5");`
		`}`

Loops

	Python	Java
Description	Uses for loops that iterate over items in a collection, and while loops for conditional repetition	Uses C-style for loops with initialization, condition, and increment, or enhanced for-each loops for collections.

Sample code	`for i in range(5):`
	`    print(i)`
	
	`while x < 10:`
	`    print(x)`
	`    x += 1`
		`for (int i = 0; i < 5; i++) {`
		`    System.out.println(i);`
		`}`
		
		`while (x < 10) {`
		`    System.out.println(x);`
		`    x++;`
		`}`

Loops

	Python	Java
Description	Uses for loops that iterate over items in a collection, and while loops for conditional repetition	Uses C-style for loops with initialization, condition, and increment, or enhanced for-each loops for collections.

Sample code	`for i in range(5):`
	`    print(i)`
	
	`while x < 10:`
	`    print(x)`
	`    x += 1`
		`for (int i = 0; i < 5; i++) {`
		`    System.out.println(i);`
		`}`
		
		`while (x < 10) {`
		`    System.out.println(x);`
		`    x++;`
		`}`

Function Definition

	Python	Java
Description	Functions are defined using the def keyword, and no return type needs to be specified.	Functions (methods) are defined with explicit return types and parameters, including access modifiers.

Sample code	`def add(a, b):`
	`    return a + b`
		`public int add(int a, int b) {`
		`    return a + b;`
		`}`

Class Definition

	Python	Java
Description	Class definitions are simpler and do not require explicit types for class members.	Classes require explicit member variable declarations and access modifiers

Sample code	`class Dog:`
	`  def __init__(self, name):`
	`  self.name = name`
	
	`  def bark(self):`
	`  print("Woof!")`
		`public class Dog {`
		`  private String name;`
		
		`  public Dog(String name) {`
		`  this.name = name;`
		`  }`
		
		`  public void bark() {`
		`  System.out.println("Woof!");`
		`  }`
		`}`

Inheritance

	Python	Java
Description	Inheritance is simpler, using parentheses to specify the parent class	Inheritance uses the extends keyword for single inheritance and implements for interfaces

Sample code	`class Animal:`
	`  def speak(self):`
	`  print("Animal speaks")`
	
	`class Dog(Animal):`
	`  def speak(self):`
	`  print("Woof!")`
		`public class Animal {`
		`  public void speak() {`
		`  System.out.println("Animal speaks");`
		`  }`
		`}`
		
		`public class Dog extends Animal {`
		`  @Override`
		`  public void speak() {`
		`  System.out.println("Woof!");`
		`  }`
		`}`

Access Modifiers

	Python	Java
Description	No explicit access modifiers (like private or public), but uses underscores to indicate protected/private attributes	Inheritance uses the extends keyword for single inheritance and implements for interfaces

Sample code	`class Example:`
	`  def __init__(self):`
	`  # Public`
	`  self.public_var = 5`
	`  # Protected`
	`  self._protected_var = 10`
	`  # Private`
	`  self.__private_var = 15`
		`public class Example {`
		`  public int publicVar = 5;  // Public`
		`  protected int protectedVar = 10;  // Protected`
		`  private int privateVar = 15;  // Private`
		`}`

Lists vs Arrays

	Python	Java
Description	Uses list for arrays, and it's dynamically resizable.	Uses Array (fixed size) or ArrayList (resizable) for arrays and lists

Sample code	`my_list = [1, 2, 3]`
	`my_list.append(4)`
		`int[] myArray = {1, 2, 3};`
		`ArrayList<Integer> myList = new ArrayList<>();`
		`myList.add(4);`

Exception Handling

	Python	Java
Description	Uses try-except for exception handling	Uses try-catch for exception handling

Sample code	`try:`
	`    result = 10 / 0`
	`except ZeroDivisionError:`
	`    print("Cannot divide by zero")`
		`try {`
		`    int result = 10 / 0;`
		`} catch (ArithmeticException e) {`
		`    System.out.println("Cannot divide by zero");`
		`}`

Importing Modules

	Python	Java
Description	Uses import to include libraries or modules	Uses import to include classes or entire packages
Sample code	`import math` `print(math.sqrt(16))`	`import java.util.Scanner;`

Main Method

	Python	Java
Description	There is no mandatory entry point, but scripts often use	The entry point is the main method, which is required for program execution.

Sample code	`if __name__ == "__main__":`
	`  main()`
		`public class Main {`
		`  public static void main(String[] args) {`
		`  System.out.println("Hello, World!");`
		`  }`
		`}`

Key Differences in I/O

Aspect	Python	Java
Input	`input()` (returns string)	`Scanner` object for various types
Type Conversion	Manual conversion required (e.g., `int()`, `float()`)	Uses methods like `nextInt()`, `nextDouble()`
Output	`print()` function	`System.out.print()` and `System.out.println()`
String Concatenation	Multiple arguments in `print()`	Uses `+` operator for concatenation

Input

	Python	Java
Description	Python uses the input() function to take user input.The input() function always returns a string, so you need to convert it to the appropriate data type (e.g., int, float) if necessary.	Java uses the Scanner class for taking user input. You need to create a Scanner object and use different methods like nextLine(), nextInt(), nextDouble(), etc., depending on the input type.

Sample code	`# Taking input from the user`
	`name = input("Enter your name: ")`
	`# Convert string to integer`
	`age = int(input("Enter your age: "))`
		`import java.util.Scanner;`
		
		`Scanner scanner = new Scanner(System.in);`
		`System.out.print("Enter your name: ");`
		`String name = scanner.nextLine();`
		`System.out.print("Enter your age: ");`
		`int age = scanner.nextInt();`

Output

	Python	Java
Description	Python uses the print() function for output.Multiple values can be printed in one print() statement, and you can also control the separator and end character.	Java uses System.out.print() and System.out.println() for output.print() outputs without a newline. println() appends a newline after the output. You can concatenate strings and variables using the + operator

Sample code	`# Output in Python`
	`print("Hello, World!")`
	`# Printing multiple values`
	`print("Your name is", name, "and your age is", age)`
		`// Output in Java`
		`System.out.println("Hello, World!");`
		`System.out.println("Your name is " + name + " and your age is " + age);`

Example: Taking input and giving output in both Python and Java

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Your name is", name, "and your age is", age)
```

```
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = scanner.nextLine();
System.out.print("Enter your age: ");
int age = scanner.nextInt();
System.out.println("Your name is " + name + " and your age is " + age);
```

Data types

- Same as Python, in Java, data types are categorized into two main groups: **primitive data types** and **reference (non-primitive) data types**

Similarities between Python and Java

Type	Python	Java
Integer	`int` (unlimited precision)	`int`, `long`
Float	`float` (double-precision)	`float`, `double`

Type	Python	Java
Boolean	`bool`	`boolean`

Type	Python	Java
String	`str` (immutable)	`String` (immutable)

Type	Python	Java
Array	Lists (mutable), `list[]`	`Array[]` (fixed size)

primitive data types in Java

Type	Size	Default Value	Description	Example
byte	1 byte	0	Stores whole numbers from -128 to 127	`byte a = 10;`
short	2 bytes	0	Stores whole numbers from -32,768 to 32,767	`short b = 5000;`
int	4 bytes	0	Stores whole numbers from -2^31 to 2^31-1	`int c = 100000;`
long	8 bytes	0L	Stores whole numbers from -2^63 to 2^63-1	`long d = 150000L;`
float	4 bytes	0.0f	Stores fractional numbers, precision up to 6-7 digits	`float e = 5.75f;`
double	8 bytes	0.0d	Stores fractional numbers, precision up to 15-16 digits	`double f = 19.99d;`
char	2 bytes	'\u0000'	Stores a single character/letter or ASCII values	`char g = 'A';`
boolean	1 bit	false	Stores `true` or `false` values	`boolean h = true;`

Reference (Non-Primitive) Data Types in Java

Type	Example	Description
String	`String s = "Hello";`	Represents sequences of characters
Array	`int[] arr = {1, 2, 3};`	Represents a fixed number of elements of the same type
Class	`Car myCar = new Car();`	Represents a blueprint for creating objects
Interface	`Runnable r = new MyClass();`	Represents abstract types that classes implement

Sample Java code:

```
// This is a single-line comment
// Importing the Scanner class for user input
import java.util.Scanner;  // Import statement

// Defining the class named "SimpleProgram"
public class SimpleProgram {  // Class definition
    // Main method: the entry point of the Java program
    public static void main(String[] args) {  // Main method declaration
        // Creating a Scanner object for input
        Scanner scanner = new Scanner(System.in);  // Object creation
        // Print a message to the console (output)
        System.out.println("Enter your name: ");  // Output statement
        // Reading user input and storing it in a variable
        String name = scanner.nextLine();  // Input statement
        // Printing a personalized greeting message
        // Output statement
        System.out.println("Hello, " + name + "! Welcome to Java.");
        // Closing the Scanner object to prevent resource leaks
        scanner.close();  // Closing input object
    }
}
```